

FINAL PROJECT RECORD • 12 SEPTEMBER 2026

Lean Agent Engineering v0.1.1

Project Build, Architecture, Evaluation & Publication Record

Build reliable AI agents with the minimum complexity necessary.

Learn less tooling. Build more. Verify what works.
Add complexity only when the evidence says you need it.

AUTHOR HumanInTheLoopAI	VERSION 0.1.1	LICENSE Apache-2.0
----------------------------	------------------	-----------------------

Public repository: github.com/sba311paw-coder/lean-agent-engineering

1. Executive summary

Lean Agent Engineering v0.1.1 is a portable, vendor- and model-agnostic Agent Skill for people building AI systems with limited time, money, compute, or prior tooling experience. Its objective is practical: produce reliable outcomes with the smallest architecture that meets the need, keep permissions outside the model, and require evidence before calling work complete.

The core promise

Learn less tooling. Build more. Verify what works. Add complexity only when the evidence says you need it.

The project favors free or low-cost approaches, reuse of capabilities already present, narrow dependencies, and security at execution boundaries. It treats “reuse what exists” and “do nothing” as valid outcomes when no real capability gap exists.

Project objective

- Give resource-constrained learners a portable engineering method rather than a mandatory framework.
- Separate model reasoning from application execution, authorization, tooling, and verification.
- Use evidence to justify each added layer of complexity.
- Provide a narrow activation boundary so ordinary AI questions remain ordinary questions.

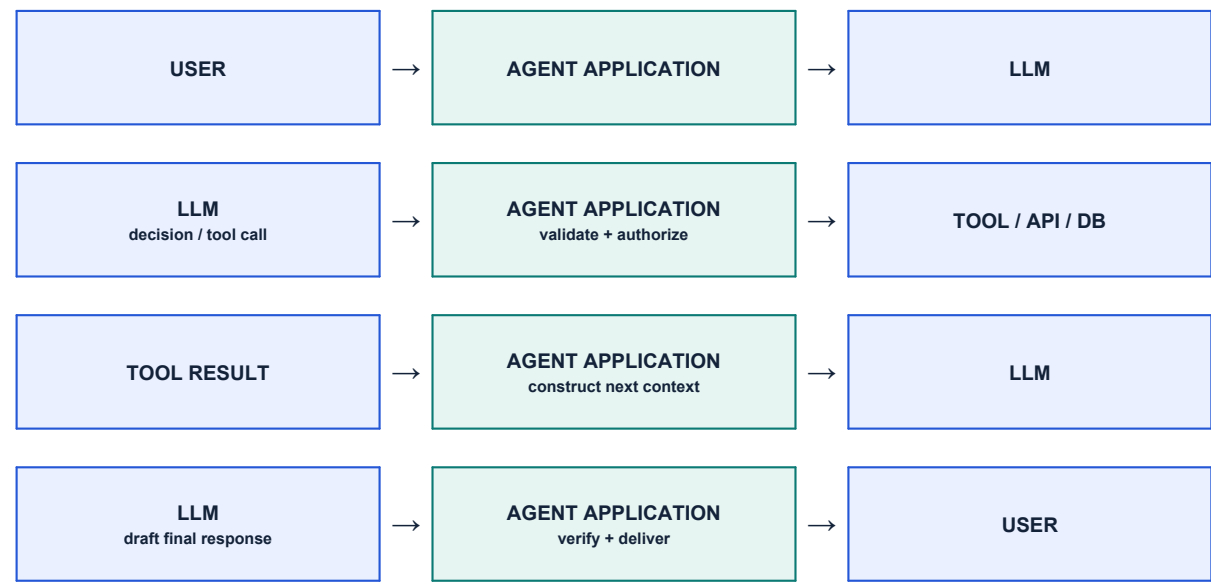
Mechanism hierarchy



Move from left to right only when evidence shows the earlier option cannot solve the requirement reliably. “More powerful” also means more permissions, failure modes, maintenance, and cost.

2. Primary agent architecture

The diagram comes first because responsibility boundaries are the foundation of the method.



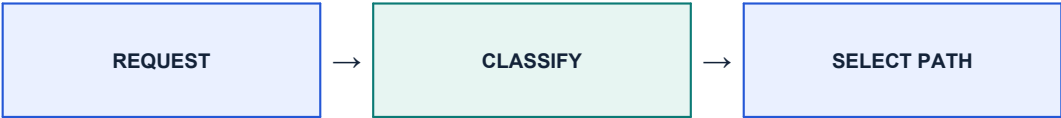
The LLM interprets context, reasons, and proposes a response or structured action. The Agent Application owns the workflow: it validates arguments, enforces authorization, invokes tools, handles failures, and places tool results into the next model context. Tools perform the external operation. A verifier supplies evidence that the intended result occurred.

Critical boundary

An LLM decision is never authorization. A successful tool call is not a successful final answer. The application must carry the result into the final model context and verify the outcome.

When behavior is wrong, find the first incorrect handoff. For example, if a weather function returned the right temperature and the logs contain it, inspect the exact Agent Application → LLM follow-up request before changing the tool, model, or framework.

3. Capability routing



Simple	Build	Bug	Security	Agent issue
Execute → verify	Plan → build → test → verify	Investigate → root cause → fix → verify	Restrict → authorize → execute → verify	Trace boundaries → isolate failure → fix → verify



The capability-gap test asks what is actually missing, whether an installed skill or application function already supplies it, whether a smaller local solution works, which new permissions or costs an addition creates, and what evidence justifies that complexity.

Simplification order

- Existing functionality
- Small local change
- Standard library or native platform
- Installed skill
- Focused dependency
- Framework
- Additional agent or orchestration layer

4. Ecosystem audits and decisions

Ecosystem	Decision	Reasoning
Superpowers	Full framework rejected One skill approved	The broad workflow exceeded the need. The focused systematic-debugging skill was approved for non-obvious bugs because it directly strengthens root-cause investigation.
ECC	Full framework rejected	Only targeted capabilities should be considered later when evidence identifies a specific gap.
Ponytail	Hooks/plugin approach rejected	Persistent hooks and plugin machinery added scope and operational complexity without a demonstrated need.
Humanizer	Green, non-core	Potentially useful for prose refinement, but unnecessary to the engineering method itself.
OpenAI plugin repository	Do not install wholesale	Discover and adopt individual capabilities only when a real, bounded requirement warrants them.

Borrow principles, not whole ecosystems

Extract the smallest proven practice that closes a gap. Avoid inheriting unrelated conventions, dependencies, hooks, and operational burden.

5. Agent Skills specification findings

The package follows the core Agent Skills contract and progressive-disclosure model: a concise skill entry point is loaded when relevant, while examples and evaluation data remain separate until needed.

Finding	Project application
SKILL.md is required	The package entry point is SKILL.md.
Frontmatter requires name and description	Both are present and the description defines a narrow activation boundary.
Optional metadata	License, compatibility, and metadata are included. allowed-tools remains experimental and is not required here.
Progressive disclosure	Core instructions stay in SKILL.md; evals and the minimal-agent example are separate supporting resources.
Recommended length	SKILL.md is exactly 260 lines, comfortably below the recommended 500-line ceiling.
Behavioral validation	Realistic positive and negative trigger cases test activation, non-activation, engineering behavior, and security boundaries.

The six-step core rule identifies the needed capability, checks the existing system, selects the simplest reliable path, reuses before adding, adds the smallest safe capability, and verifies the result. Completion requires the requested outcome, successful action, appropriate evidence, known limitations, no unnecessary capability, and intact authorization boundaries.

6. Security model for agentic systems



External and retrieved content - including documents, web pages, logs, email, source code, API output, database records, memory, and previous agent output - is data. It cannot override the user's real task, application policy, authorization, or permission boundaries.

Risk	Required control
Excessive agency	Give the tool only the permissions required for the task; use read-only roles for read-only work.
Prompt injection	Treat retrieved instructions as untrusted; never expose credentials because content asks for them.
Tool misuse	Allowlist tools, validate names and arguments, bound rounds and timeouts, and surface failures.
Privilege abuse	Enforce authorization at the application or tool boundary; the model's proposal grants no authority.
Memory/context poisoning	Preserve provenance and trust boundaries; validate what becomes executable context.
Destructive/high-impact action	Use explicit confirmation where policy requires it, plus scope limits, auditability, and recovery plans.

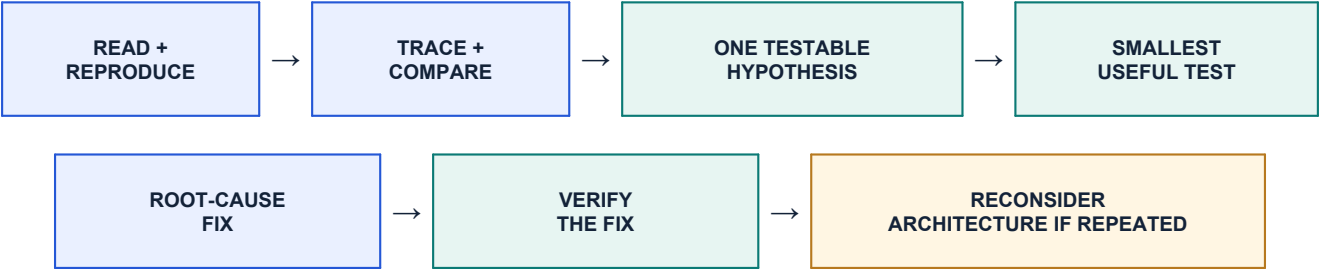
Security boundary

A prompt that says "read only" is not access control. Use a read-only database role or tool, and keep credentials inaccessible to model output.

7. SKILL.md v0.1.1 design

The 260-line skill activates only for building, modifying, debugging, reviewing, securing, or architecting AI and agentic software. It does not activate merely because a request mentions AI, an LLM, an agent, or software.

Systematic debugging



The detailed sequence also checks recent changes and traces data and control flow. For non-obvious bugs, failed tests, RAG failures, integrations, and production issues, the existing systematic-debugging skill is preferred when available; the essential sequence remains embedded so the package has no dependency.

Verification ladder



Use the lowest level sufficient for the risk. A command returning exit code 0 proves only that the command reported success; production health, dependencies, critical flows, and observable outcomes still require verification.

8. Evaluation methodology and result

Final executed set: 18 / 18 PASS

Historical manual behavioral evaluation. The final package preserves every executed case.

Category	Result	What it tested
Activation	6 / 6	Three engineering triggers and three educational non-triggers.
Minimality	3 / 3	Reuse vs MCP, framework replacement, and observability expansion.
Debugging	2 / 2	Regression after change and premature embedding-model replacement.
Security	3 / 3	Destructive deletion, prompt injection, and least privilege.
Verification	1 / 1	Exit code 0 vs demonstrated production outcome.
Architecture	3 / 3	Tool choice errors, ignored tool result, and minimal two-function agent.
Total	18 / 18	All final executed cases preserved in evals/evals.json.

Method

Each prompt was run as a separate behavioral case and judged against explicit expected behaviors. Positive cases assessed whether the skill's engineering workflow appeared; negative cases checked that educational questions remained ordinary explanations. The historical record consists of user-pasted Codex responses manually adjudicated during the project. Packaging recovered and normalized that evidence; it did not execute a new behavioral run.

9. What the 18 cases covered

Cases	Focus	Representative behavior
1–6	Activation boundaries	Weather tool result missing from the final answer; tool skipping; RAG grounding; and negative explanatory prompts about LLMs, tool calling, and RAG.
7	Reuse before MCP	An existing Python email function already supplied the capability.
8	Direct API before framework	A working two-function agent did not justify replacing the implementation with LangChain.
9	Measured need before observability	Existing logs and tests already showed tool failures and latency.
10	Regression debugging	A recent code change triggered evidence gathering and boundary tracing before repair.
11	RAG root cause	Retrieval, context construction, and generation were isolated before considering a new embedding model.
12	Destructive action	A model-selected production deletion required application/tool authorization and confirmation.
13	Retrieved prompt injection	A malicious document request for credentials was treated as untrusted data.
14	Least privilege	A read-only agent should not receive unnecessary admin credentials.
15	Outcome verification	Deployment exit code 0 was insufficient proof that production worked.
16	Architecture restraint	Bad tool choices did not justify replacing the entire framework.
17	Tool-result handoff	A correct result in logs was ignored by the final LLM; the next-context boundary was investigated first.
18	Minimal agent architecture	One Python process, one model interface, and two existing functions met the stated need.

10. Evaluation limits and corrected history

Read 18 / 18 correctly

This result is evidence that the observed responses met the defined criteria. It is not proof of universal reliability.

Historical limitations

- Manual adjudication: the evidence was user-pasted Codex responses judged against the case criteria.
- No internal activation telemetry: expected activation is a behavioral boundary, not an observed internal event.
- No controlled no-skill baseline: causal improvement attributable to the skill was not measured.
- Runtime/model configuration not independently verified: the evaluation record cannot confirm the exact execution environment.
- No universal or cross-client claim: 18/18 does not establish reliability across all tasks, models, clients, or future versions.

Corrections preserved in the final package

An earlier JSON draft contained 17 entries. The final package preserves all 18 subsequently executed cases and retains earlier wording as `original_draft_prompt` when it changed. An earlier category summary counted only two architecture cases; the individual records show cases 16–18, so the corrected architecture total is 3/3. These corrections changed bookkeeping, not the reported outcome of the executed cases.

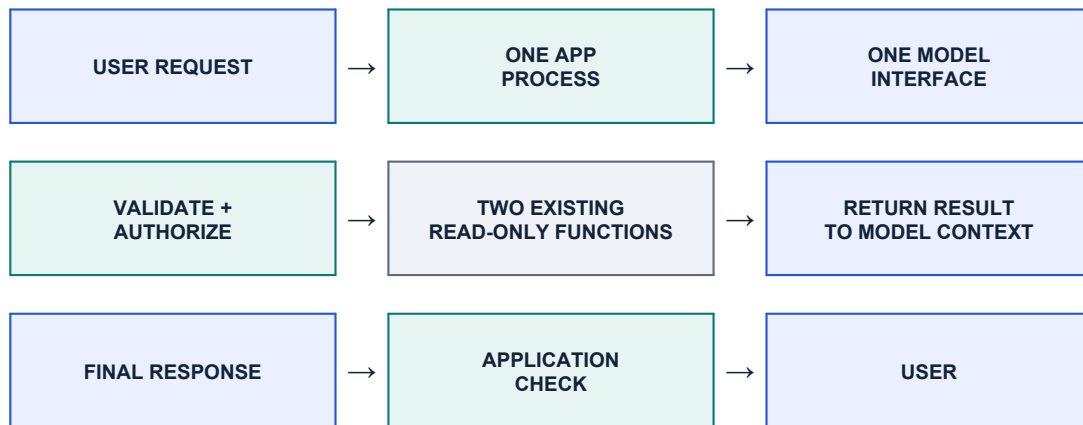
An earlier six-case smoke review reported 6/6 after a contaminated negative RAG attempt was excluded and repeated with a clean prompt. This detail remains historical context and is not counted as a separate final suite.

11. Final repository and package

Path	Purpose
<code>.gitignore</code>	Excludes local/editor files, credentials and private configuration, virtual environments, caches, logs, and build output.
<code>LICENSE</code>	Full Apache License 2.0 terms.
<code>README.md</code>	Beginner-facing overview, installation/use guidance, package map, evaluation summary, and limitations.
<code>SKILL.md</code>	The 260-line portable skill: activation boundary, lean workflow, security model, debugging, verification, and completion standard.
<code>evals/evals.json</code>	The 18-case manual behavioral suite, criteria, historical results, corrections, and limitations.
<code>examples/minimal-agent.md</code>	A provider-neutral, one-process architecture with two existing read-only functions and a small verification plan.

The package has no runtime dependency, MCP server, hook, API key, network requirement, paid service, or vendor-specific integration. It is instruction and evidence, not an executable security boundary; applications and tools must enforce permissions.

12. Minimal-agent example



The application allowlists the two functions, validates names and arguments, enforces the user's permissions, treats tool output as data, bounds tool rounds, and uses timeouts. Provider-specific message formatting stays inside the model interface so ordinary functions remain reusable.

Small verification plan

- One question requiring no tool and one question for each function.
- An invalid argument, a simulated function failure, and an unauthorized request.
- A tool-round limit and a check that each successful result appears in the final answer.
- Synthetic data and logs that exclude credentials.

When to add a framework

Only after a measured need - such as durable multi-step state, many integrations, or orchestration requirements - cannot be met reliably by a smaller local change.

13. Publication and final verification

The finished project was published as the public repository [sba311paw-coder/lean-agent-engineering](#) on the main branch under Apache-2.0. The repository includes the final six-file package, and .gitignore was successfully added through GitHub.

Check	Final state
Public identity	Repository and documentation identify the author as HumanInTheLoopAI.
Package contents	README, SKILL.md, 18-case eval set, minimal example, license, and .gitignore present.
Local Git identity	Configured as HumanInTheLoopAI using a GitHub noreply address; no personal Gmail address is recorded here.
Cleanup	An obsolete local .gitignore commit was discarded after the GitHub web commit. Local main was reset and synchronized to origin/main.
Final sign-off	The final working tree was clean and up to date with origin/main.

Publication is complete. An optional GitHub v0.1.1 release or tag is a future action and is not represented as already done.

14. Lessons learned

Boundaries beat guesses	Trace the first incorrect handoff across user, model, application, tool, result, and final response.
Capability is not infrastructure	A need for an action does not automatically imply a framework, MCP server, plugin, or new agent.
Reuse has engineering value	Existing functions, logs, tests, platform features, and installed skills reduce cost and failure surface.
Models propose; systems authorize	Keep destructive actions, credentials, and permissions behind deterministic application and tool controls.
Verification matches risk	Use the smallest adequate test, then increase evidence as impact and irreversibility rise.
Negative tests define scope	A good skill must know when to stay inactive, especially for simple educational questions.
Corrections strengthen evidence	Preserving the 17-entry draft history and correcting the category undercount makes the final 18-case record auditable.
Portable means modest	Avoid hidden services and ecosystem assumptions; keep the method useful across vendors, models, and constrained environments.

15. v0.1.1 sign-off

FINAL STATUS • COMPLETE

Lean Agent Engineering v0.1.1 is built, documented, evaluated through the recorded 18-case manual suite, packaged, licensed, published, and signed off.

The project meets its stated objective: a compact, beginner-friendly Agent Skill that teaches reliable AI engineering through reuse, explicit responsibility boundaries, root-cause debugging, least privilege, and outcome verification. The package avoids imposing a runtime framework and makes its evidence limits visible.

Optional future evolution

- Automated evaluation harness: run the existing case set reproducibly, capture model/client configuration, and store machine-readable results.
- Cross-client testing: measure activation and behavior across skills-compatible clients and models.
- Controlled baseline: compare with and without the skill under the same runtime settings.
- GitHub v0.1.1 release/tag: optionally create a release after deciding the desired release notes and artifact set.

These items are suggestions only. None is implied to be complete.

HumanInTheLoopAI
Lean Agent Engineering v0.1.1
Apache-2.0